



ORACLE
NetSuite

SOAP Web Services to REST Web Services Upgrade Guide

2025.2

December 17, 2025



Copyright © 2005, 2025, Oracle and/or its affiliates.

This software and related documentation are provided under a license agreement containing restrictions on use and disclosure and are protected by intellectual property laws. Except as expressly permitted in your license agreement or allowed by law, you may not use, copy, reproduce, translate, broadcast, modify, license, transmit, distribute, exhibit, perform, publish, or display any part, in any form, or by any means. Reverse engineering, disassembly, or decompilation of this software, unless required by law for interoperability, is prohibited.

The information contained herein is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

If this is software, software documentation, data (as defined in the Federal Acquisition Regulation), or related documentation that is delivered to the U.S. Government or anyone licensing it on behalf of the U.S. Government, then the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software," "commercial computer software documentation," or "limited rights data" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

This software or hardware is developed for general use in a variety of information management applications. It is not developed or intended for use in any inherently dangerous applications, including applications that may create a risk of personal injury. If you use this software or hardware in dangerous applications, then you shall be responsible to take all appropriate fail-safe, backup, redundancy, and other measures to ensure its safe use. Oracle Corporation and its affiliates disclaim any liability for any damages caused by use of this software or hardware in dangerous applications.

Oracle®, Java, and MySQL are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

This software or hardware and documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

If this document is in public or private pre-General Availability status:

This documentation is in pre-General Availability status and is intended for demonstration and preliminary use only. It may not be specific to the hardware on which you are using the software. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to this documentation and will not be responsible for any loss, costs, or damages incurred due to the use of this documentation.

If this document is in private pre-General Availability status:

The information contained in this document is for informational sharing purposes only and should be considered in your capacity as a customer advisory board member or pursuant to your pre-General Availability trial agreement only. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, timing, and pricing of any features or functionality described in this document may change and remains at the sole discretion of Oracle.

This document in any form, software or printed matter, contains proprietary information that is the exclusive property of Oracle. Your access to and use of this confidential material is subject to the terms and conditions of your Oracle Master Agreement, Oracle License and Services Agreement, Oracle PartnerNetwork Agreement, Oracle distribution agreement, or other license agreement which has been executed by you and Oracle and with which you agree to comply. This document and information contained herein may not be disclosed, copied, reproduced, or distributed to anyone outside Oracle without prior written consent of Oracle. This document is not part of your license agreement nor can it be incorporated into any contractual agreement with Oracle or its subsidiaries or affiliates.

Documentation Accessibility

For information about Oracle's commitment to accessibility, visit the Oracle Accessibility Program website at <https://www.oracle.com/corporate/accessibility>.

Access to Oracle Support

Oracle customers that have purchased support have access to electronic support through My Oracle Support. For information, visit <http://www.oracle.com/pls/topic/lookup?ctx=acc&id=info> or visit <http://www.oracle.com/pls/topic/lookup?ctx=acc&id=trs> if you are hearing impaired.

Sample Code

Oracle may provide sample code in SuiteAnswers, the Help Center, User Guides, or elsewhere through help links. All such sample code is provided "as is" and "as available", for use only with an authorized NetSuite Service account, and is made available as a SuiteCloud Technology subject to the SuiteCloud Terms of Service at www.netsuite.com/tos.

Oracle may modify or remove sample code at any time without notice.

No Excessive Use of the Service

As the Service is a multi-tenant service offering on shared databases, Customer may not use the Service in excess of limits or thresholds that Oracle considers commercially reasonable for the Service. If Oracle reasonably concludes that a Customer's use is excessive and/or will cause immediate or ongoing performance issues for one or more of Oracle's other customers, Oracle may slow down or throttle Customer's excess use until such time that Customer's use stays within reasonable limits. If Customer's particular usage pattern requires a higher limit or threshold, then the Customer should procure a subscription to the Service that accommodates a higher limit and/or threshold that more effectively aligns with the Customer's actual usage pattern.

Beta Features

This software and related documentation are provided under a license agreement containing restrictions on use and disclosure and are protected by intellectual property laws. Except as expressly permitted in your license agreement or allowed by law, you may not use, copy, reproduce, translate, broadcast, modify, license, transmit, distribute, exhibit, perform, publish, or display any part, in any form, or by any means. Reverse engineering, disassembly, or decompilation of this software, unless required by law for interoperability, is prohibited.

The information contained herein is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

If this is software or related documentation that is delivered to the U.S. Government or anyone licensing it on behalf of the U.S. Government, then the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software" or "commercial computer software documentation" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

This software or hardware is developed for general use in a variety of information management applications. It is not developed or intended for use in any inherently dangerous applications, including applications that may create a risk of personal injury. If you use this software or hardware in dangerous applications, then you shall be responsible to take all appropriate fail-safe, backup, redundancy, and other measures to ensure its safe use. Oracle Corporation and its affiliates disclaim any liability for any damages caused by use of this software or hardware in dangerous applications.

Oracle and Java are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

This software or hardware and documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

This documentation is in pre-General Availability status and is intended for demonstration and preliminary use only. It may not be specific to the hardware on which you are using the software. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to this documentation and will not be responsible for any loss, costs, or damages incurred due to the use of this documentation.

The information contained in this document is for informational sharing purposes only and should be considered in your capacity as a customer advisory board member or pursuant to your pre-General Availability trial agreement only. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, and timing of any features or functionality described in this document remains at the sole discretion of Oracle.

This document in any form, software or printed matter, contains proprietary information that is the exclusive property of Oracle. Your access to and use of this confidential material is subject to the terms and conditions of your Oracle Master Agreement, Oracle License and Services Agreement, Oracle PartnerNetwork Agreement, Oracle distribution agreement, or other license agreement which has been executed by you and Oracle and with which you agree to comply. This document and information contained herein may not be disclosed, copied, reproduced, or distributed to anyone outside Oracle without prior written consent of Oracle. This document is not part of your license agreement nor can it be incorporated into any contractual agreement with Oracle or its subsidiaries or affiliates.

Table of Contents

SOAP Web Services to REST Web Services Upgrade Guide	1
SOAP Web Services and REST Web Services Operations	4
add	4
checkAsyncStatus	5
delete	6
get	7
getAccountGovernanceInfo	9
getAll	10
getAsyncResult	12
getCurrencyRate	13
getIntegrationGovernanceInfo	15
getItemAvailability	16
getList	18
getServerTime	20
initialize / initializeList	20
search	22
searchMoreWithId	24
update	27
updateInviteeStatus	29
upsert	29
SOAP Web Services and REST Web Services Supported Records	32

SOAP Web Services to REST Web Services Upgrade Guide

The NetSuite REST web services provide an integration channel that extends the capabilities of SuiteTalk. REST web services provide a REST-based interface for interacting with NetSuite.

REST web services provide a lightweight integration option, where no coding and script deployment is needed on the server side, and which is suitable for mobile devices.

REST web services also support strong authentication with OAuth 2.0 and TBA.

 **Note:** The preferred authentication method is OAuth 2.0.

Using the REST API, fewer calls may be required to accomplish a business flow. Therefore the overall performance may be better than with SOAP web services. REST web services also supports asynchronous request processing. The asynchronous execution of requests is useful for long-running requests. With asynchronous requests, you send a request to REST web services, where it is placed in a processing queue and handled asynchronously with other requests.

This guide provides you with the resources to help you upgrade your SOAP web services integration to REST web services. To start learning about REST web services in general, see the following topics:

- [Overview of SuiteTalk REST Web Services](#)
- [Setting Up Authentication](#)
- [REST Web Services Request Processing](#)
- [Using Postman with REST Web Services](#)

The guide provides a side by side comparison of SOAP and REST operations, and provides information about how to perform actions in REST to get the same functionality as in SOAP.

The second part of the guide provides an overview of the record types available in SOAP and REST.

- [SOAP Web Services and REST Web Services Operations](#)
- [SOAP Web Services and REST Web Services Supported Records](#)

Comparison of SOAP Web Services and REST Web Services Operations

The following table maps SOAP operations to their REST equivalents. Click the links for detailed information about SOAP operations and their REST equivalents.

SOAP Operation	REST Operation	Notes
add	POST	
addList	—	List operations are currently not supported in REST web services.
attach / detach	—	The attach / detach operation is currently not supported in REST web services. You can use SuiteScript to define or remove a relationship between two records. For more information, see the help topics record.attach(options) and record.detach(options) .

SOAP Operation	REST Operation	Notes
changeEmail	—	Login through user credentials is not supported in REST web services.
changePassword	—	Login through user credentials is not supported in REST web services.
checkAsyncStatus	GET	You can use the async service to retrieve the status of a single async operation.
delete	DELETE	
deleteList	—	List operations are currently not supported in REST web services.
get	GET	
getAccountGovernanceInfo	governanceLimits	
getAll	GET	
getAsyncResult	—	You can use the async service to retrieve the result of a single async operation.
getBudgetExchangeRate	—	The budget exchange rate record is currently not available in REST web services. You can interact with the budget exchange rate record using SuiteScript. For details, see the help topic Budget Exchange Rate .
getCurrencyRate	GET	You can achieve the equivalent in REST by sending a GET request for a specific currency rate record.
getCustomizationId		In REST web services, you can return data about your customization objects by sending a request to the metadata catalog. For information, see the help topic Getting Metadata .
getDataCenterUrls	—	REST web services only use account-specific domains.
getDeleted	—	This functionality is not available in REST web services. You can retrieve information about deleted records by using the Deleted Record search type or a SuiteQL query. For information, see the help topic Searching for Deleted Records .
getIntegrationGovernanceInfo	governanceLimits	
getItemAvailability		You can achieve the equivalent in REST by sending a GET request for a specific record.
getList	GET	You can retrieve a list of records of the same record type in REST.
getPostingTransactionSummary	—	You can search for financial reports and summaries in the UI. For information, see the help topic Financial Statements Overview .
getSavedSearch	—	Saved Search isn't supported in REST web services. REST web services supports SuiteAnalytics Datasets. For more information, see the help topic Working with SuiteAnalytics Datasets in REST Web Services .

SOAP Operation	REST Operation	Notes
getSelectValue	—	The getSelectValue operation is currently not supported in REST web services.
getServerTime	serverTime	
initialize / initializeList		In REST web services, you can transform a record from one type into another, using data from an existing record. This is the equivalent of the initialize SOAP operation. For more information, see the help topic Transforming Records .
search	GET	
searchMoreWithId		In REST web services, you can use collection paging to move between pages of search results.
update	PATCH	
updateList	—	List operations are currently not supported in REST web services.
updateInviteeStatus	PATCH	You can achieve the equivalent in REST by sending a PATCH request for a specific record.
updateInviteeStatusList	—	List operations are currently not supported in REST web services. You can update calendar events one by one. For information, see updateInviteeStatus .
upsert	PUT	
upsertList	—	List operations are currently not supported in REST web services.

SOAP Web Services and REST Web Services Operations

This topic provides a comparison between SOAP web services operations and their REST web services equivalent. The information in this topic helps you upgrade your SOAP integration to a REST integration. See the individual topics for details on how you can achieve your SOAP web services results in REST web services.

- [add](#)
- [checkAsyncStatus](#)
- [delete](#)
- [get](#)
- [getAccountGovernanceInfo](#)
- [getAll](#)
- [getAsyncResult](#)
- [getCurrencyRate](#)
- [getIntegrationGovernanceInfo](#)
- [getItemAvailability](#)
- [getList](#)
- [getServerTime](#)
- [initialize / initializeList](#)
- [search](#)
- [update](#)
- [updateInviteeStatus](#)
- [upsert](#)

add

The add operation in SOAP web services is used to add a new instance of a record in NetSuite. It is similar to the addList operation except that it allows only one record to be added at a time. Note that to prevent duplicate records, you should use the alternate [upsert](#) and [upsertList](#) operations along with external IDs to add records to NetSuite.

SOAP Request

In the following example, a single customer record is added.

```

1 <soap:Body>
2 <platformMsgs:add>
3   <platformMsgs:record xsi:type="listRel:Customer">
4     <listRel:companyName>Shutter Fly, Inc</listRel:companyName>
5     <listRel:unsubscribe>>false</listRel:unsubscribe>
6   </platformMsgs:record>
7 </platformMsgs:add>
8 </soap:Body>

```

SOAP Response

In the response, notice that the internal ID for the record added is returned.

```

1 <soapenv:Body>
2 <addResponse xmlns="urn:messages_2017_1.platform.webservices.netsuite.com">
3 <writeResponse xmlns="urn:messages_2017_1.platform.webservices.netsuite.com">
4   <ns1:status isSuccess="true" xmlns:ns1="urn:core_2017_1.platform.webservices.netsuite.com"/>
5   <baseRef internalId="979" type="customer" xsi:type="ns2:RecordRef" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xmlns:ns2="urn:core_2017_1.platform.webservices.netsuite.com"/>
6 </writeResponse>
7 </addResponse>
8 </soapenv:Body>

```

In REST web services, you can create a new record using the POST HTTP method. The POST method expects a request body (a JSON object) that conforms to the record's metadata schema and contains values for at least each required field of the given record type.

REST Request

In the following example, a single customer record is added.

```

1 POST https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer
2 { "entityid": "New Customer", "companyname": "Shutter Fly, Inc", "subsidiary": { "id": "1" } }

```

REST Response

The following is an excerpt from the response headers of a successful operation, with no body content returned (HTTP Code 204). The URL of the newly created record is given in the Location header of the response.

```

1 Date Fri, 04 Jan 2025 08:50:20 GMT
2 Location https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer/647
3 Content-Type application/json

```

checkAsyncStatus

To check the status of an asynchronous SOAP web services job, you use the checkAsyncStatus operation. When you use this operation, you reference a job Id. In response, the system returns the job's status, among other details.

SOAP Request

The following example shows a checkAsyncStatus request.

```

1 <soap:Body>
2   <platformMsgs:checkAsyncStatus>
3     <platformMsgs:jobId>ASYNCSOAPSERVICES_563214_053120061943428686160042948_4bee0685
4   </platformMsgs:jobId>
5   </platformMsgs:checkAsyncStatus>
6 </soap:Body>=

```

SOAP Response

```

1 <soapenv:Body>
2 <checkAsyncStatusResponse xmlns="urn:messages_2017_1.platform.webservices.netsuite.com">
3   <asyncStatusResult xmlns="urn:core_2017_1.platform.webservices.netsuite.com">
4     <jobId>ASYNCSERVICES_563214_053120061943428686160042948_4bee0685</jobId>
5     <status>pending</status>
6     <percentCompleted>0.0</percentCompleted>
7     <estRemainingDuration>0.0</estRemainingDuration>
8   </asyncStatusResult>
9 </checkAsyncStatusResponse>
10 </soapenv:Body>

```

In REST web services, you can use the async service to retrieve the status of a single async operation. For detailed information about asynchronous processing in REST and for more examples, see the help topic [REST Web Services Request Processing](#).

REST Request

You can send a request similar to the following to get the task ID of the asynchronous request: GET <https://demo123.suitetalk.api.netsuite.com/services/rest/async/v1/job/1/task/>

Then you can send a request similar to the following to see the status of a completed asynchronous request: GET <https://demo123.suitetalk.api.netsuite.com/services/rest/async/v1/job/1/task/1>

REST Response

The following example shows the returned result.

```

1 {
2   "links": [
3     {
4       "rel": "related",
5       "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/async/v1/job/1/task/1/result"
6     },
7     {
8       "rel": "self",
9       "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/async/v1/job/1/task/1"
10    },
11  ],
12  "completed": true,
13  "endTime": "2020-12-09T12:04:40Z",
14  "id": "1",
15  "progress": "succeeded",
16  "startTime": "2020-12-09T12:04:28Z"
17 }

```

delete

The delete operation is used to delete an instance of a record. It enables you to delete one record to per request.

SOAP Request

In the following example, a single cash sale record is deleted.

```

1 <soap:Body>
2   <delete xmlns="urn:messages_2017_1.platform.webservices.netsuite.com">
3     <baseRef type="cashSale" internalId="1745" xsi:type="q1:RecordRef" xmlns:q1="urn:core_2017_1.platform.webservices.netsuite.com"/>
4     <deletionReason>
5       <deletionReasonCode internalId="3" xmlns="urn:core_2017_1.platform.webservices.netsuite.com"/>
6       <deletionReasonMemo xmlns="urn:core_2017_1.platform.webservices.netsuite.com">Per John in Accounting</deletionReasonMemo>
7     </deletionReason>
8   </delete>
9 </soap:Body>

```

SOAP Response

The following response confirms that the record has been deleted.

```

1 <soapenv:Body>
2   <deleteResponse xmlns="urn:messages_2017_1.platform.webservices.netsuite.com">
3     <writeResponse>
4       <platformCore:status isSuccess="true" xmlns:platformCore="urn:core_2017_1.platform.webservices.netsuite.com"/>
5       <baseRef xsi:type="platformCore:RecordRef" type="cashSale" internalId="1745" xmlns:platformCore="urn:core_2017_1.platform.webservices.netsuite.com"/>
6     </writeResponse>
7   </deleteResponse>
8 </soapenv:Body>

```

In REST web services, you can delete a record instance using the DELETE HTTP method. To delete a record instance, you need to specify the record type and the instance identifier.

REST Request

In the following example, a single cash sale record is deleted.

```

1 | DELETE https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/cashSale/107

```

REST Response

The following is an excerpt from the response headers of a successful operation, with no body content returned (HTTP Code 204).

```

1 | Date Thu, 14 Feb 2024 15:38:21 GMT
2 | Content-Type application/json

```

get

In SOAP web services, the get operation is used to retrieve a record by providing the unique id that identifies that record.

SOAP Request

In the following example, a single customer record is retrieved. Note that the internal ID for the specific instance of the record and the record type (customer) must be specified.

```

1 <soapenv:Body>
2   <get xmlns="urn:messages_2025_1.platform.webservices.netsuite.com">
3     <baseRef internalId="107" type="customer" xsi:type="ns7:RecordRef" xmlns:ns7="urn:core_2025_1.platform.webservices.netsuite.com"/>
4   </get>
5 </soapenv:Body>

```

SOAP Response

The following example shows a possible response.

```

1 <soapenv:Body>
2   <getResponse xmlns="urn:messages_2025_1.platform.webservices.netsuite.com">
3     <readResponse>
4       <platformCore:status isSuccess="true" xmlns:platformCore="urn:core_2025_1.platform.webservices.netsuite.com"/>
5       <record internalId="107" xsi:type="listRel:Customer" xmlns:listRel="urn:relationships_2025_1.lists.webservices.netsuite.com">
6         <listRel:customForm internalId="3" xmlns:platformCore="urn:core_2025_1.platform.webservices.netsuite.com">
7           <platformCore:name>Customer_Preferred=T_Store=F</platformCore:name>
8         </listRel:customForm>
9         <listRel:entityId>Tim Harris</listRel:entityId>
10        <listRel:isPerson>false</listRel:isPerson>
11        <listRel:companyName>Glenrock General Hospital</listRel:companyName>
12        <listRel:entityStatus internalId="13" xmlns:platformCore="urn:core_2025_1.platform.webservices.netsuite.com">
13          <platformCore:name>CUSTOMER-Closed Won</platformCore:name>
14        </listRel:entityStatus>
15
16        ...[more fields]...
17
18        <listRel:dateCreated>2017-09-04T15:59:15.000-07:00</listRel:dateCreated>
19        <listRel:customFieldList xmlns:platformCore="urn:core_2025_1.platform.webservices.netsuite.com">
20          <platformCore:customField internalId="59" scriptId="custentity_633637_asubmit" xsi:type="platformCore:BooleanCustomFieldRef">
21            <platformCore:value>false</platformCore:value>
22          </platformCore:customField>
23          <platformCore:customField internalId="58" scriptId="custentity_633637_blood" xsi:type="platformCore:BooleanCustomFieldRef">
24            <platformCore:value>false</platformCore:value>
25          </platformCore:customField>
26        </listRel:customFieldList>
27      </record>
28    </readResponse>
29  </getResponse>
30 </soapenv:Body>

```

REST Request

In REST web services, you also use the get operation to retrieve a record instance.

In the following example, a single customer record is retrieved.

```

1 | GET https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer/107

```

REST Response

The following is a shortened example of the body of the response.

```

1 {
2   "links": [
3     {
4       "rel": "self",

```

```

5      "href": "http://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer/107"
6    }
7  ],
8  "accessRole": {
9    "links": [
10     {
11       "rel": "self",
12       "href": "http://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/"
13     }
14   ],
15   "id": "14",
16   "refName": "Customer Center"
17 },
18 "accessTabChanged": false,
19 "addressbook": {
20   "links": [
21     {
22       "rel": "self",
23       "href": "http://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer/107/addressbook"
24     }
25   ]
26 },
27 "autoName": true,
28 "balance": 0,
29 "billPay": false,
30 "bulkmerge": {
31   "links": [
32     {
33       "rel": "self",
34       "href": "http://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer/107/bulkmerge"
35     }
36   ]
37 },

```

getAccountGovernanceInfo

In SOAP web services, you can use the `getAccountGovernanceInfo` operation to retrieve the account concurrency limit and the unallocated concurrency limit.

If an integration has insufficient concurrency limit allocated to it, it can use the `getAccountGovernanceInfo` operation. With the retrieved information the integration can determine whether there is sufficient unallocated concurrency limit for the account and can suggest the administrator to increase the concurrency limit allocated to the integration.

SOAP Request

The following example shows the usage of the `getAccountGovernanceInfo` operation.

```

1 <soapenv:Body>
2   <getAccountGovernanceInfo/>
3 </soapenv:Body>

```

SOAP Response

The following example shows the result of a `getAccountGovernanceInfo` request.

```

1 <soapenv:Body>
2   <getAccountGovernanceInfoResponse>
3     <platformCore:getAccountGovernanceInfoResult>
4       <platformCore:status isSuccess="true"/>

```

```

5         <platformCore:accountConcurrencyLimit>5</platformCore:accountConcurrencyLimit>
6         <platformCore:unallocatedConcurrencyLimit>3</platformCore:unallocatedConcurrencyLimit>
7     </platformCore:getAccountGovernanceInfoResult>
8 </getAccountGovernanceInfoResponse>
9 </soapenv:Body>

```

REST Request

In REST web services, you use the `governanceLimits` operation to retrieve information about concurrency limits.

The `governanceLimits` operation does not have any parameters and returns (1) the account concurrency limit, which specifies the maximum number of concurrent connections allowed for a given account, (2) the unallocated concurrency limit, which indicates a limit for all integrations without a specifically defined limit, and (3) the limit type.

The following example shows the usage of the `governanceLimits` operation.

```

1 | GET https://demo123.suitetalk.api.netsuite.com/services/rest/system/v1/governanceLimits

```

REST Response

The `governanceLimits` operation can return three responses based on the concurrency limit type. The following example shows a response with information about the account limit.

```

1 | {
2 |   "accountConcurrencyLimit": 5,
3 |   "accountUnallocatedConcurrencyLimit": 3,
4 |   "integrationLimitType": "accountLimit"
5 | }

```

getAll

In SOAP web services, the `getAll` operation is used to retrieve a list of all records of the specified record type.

SOAP Request

The following sample shows how to retrieve the list of all currency records.

```

1 <soapenv:Body>
2   <getAll xmlns="urn:messages_2025_1.platform.webservices.netsuite.com">
3     <record recordType="currency"/>
4   </getAll>
5 </soapenv:Body>

```

SOAP Response

The following sample shows the response to the previous request.

```

1 <soapenv:Body>
2   <getAllResponse xmlns="urn:messages_2025_1.platform.webservices.netsuite.com">

```

```

3      <platformCore:getAllResult xmlns:platformCore="urn:core_2025_1.platform.webservices.netsuite.com">
4      <platformCore:status isSuccess="true"/>
5      <platformCore:totalRecords>2</platformCore:totalRecords>
6      <platformCore:recordList>
7      <platformCore:record internalId="1" xsi:type="listAcct:Currency" xmlns:listAcct="urn:accounting_2025_1.lists.webser
vices.netsuite.com">
8          <listAcct:name>US USD</listAcct:name>
9          <listAcct:symbol>USD</listAcct:symbol>
10         <listAcct:isBaseCurrency>true</listAcct:isBaseCurrency>
11         <listAcct:isInactive>false</listAcct:isInactive>
12         <listAcct:overrideCurrencyFormat>false</listAcct:overrideCurrencyFormat>
13         <listAcct:displaySymbol>$</listAcct:displaySymbol>
14         <listAcct:symbolPlacement>_beforeNumber</listAcct:symbolPlacement>
15         <listAcct:locale>_unitedStatesEnglish</listAcct:locale>
16         <listAcct:formatSample>$1,234.56</listAcct:formatSample>
17         <listAcct:exchangeRate>1.0</listAcct:exchangeRate>
18         <listAcct:currencyPrecision>_two</listAcct:currencyPrecision>
19     </platformCore:record>
20     <platformCore:record internalId="2" xsi:type="listAcct:Currency" xmlns:listAcct="urn:accounting_2025_1.lists.webser
vices.netsuite.com">
21         <listAcct:name>UK GBP</listAcct:name>
22         <listAcct:symbol>GBP</listAcct:symbol>
23         <listAcct:isBaseCurrency>true</listAcct:isBaseCurrency>
24         <listAcct:isInactive>false</listAcct:isInactive>
25         <listAcct:overrideCurrencyFormat>false</listAcct:overrideCurrencyFormat>
26         <listAcct:displaySymbol>&#xA3;</listAcct:displaySymbol>
27         <listAcct:symbolPlacement>_beforeNumber</listAcct:symbolPlacement>
28         <listAcct:locale>_unitedKingdomEnglish</listAcct:locale>
29         <listAcct:formatSample>&#xA3;1,234.56</listAcct:formatSample>
30         <listAcct:exchangeRate>0.6</listAcct:exchangeRate>
31         <listAcct:currencyPrecision>_two</listAcct:currencyPrecision>
32     </platformCore:record>
33 </platformCore:recordList>
34 </platformCore:getAllResult>
35 </getAllResponse>
36 </soapenv:Body>

```

REST Request

In REST web services, you can obtain the list of all records of a record type by sending an HTTP GET request as in the following example.

```
1 GET https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/currency
```

REST Response

The following example shows the result of the previous REST request.

```

1 {
2   "links": [
3     {
4       "rel": "self",
5       "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/currency/"
6     }
7   ],
8   "count": 5,
9   "hasMore": false,
10  "items": [
11    {
12      "links": [
13        {
14          "rel": "self",
15          "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/currency/2"
16        }
17      ],

```



```

18     "id": "2"
19   },
20   {
21     "links": [
22       {
23         "rel": "self",
24         "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/currency/3"
25       }
26     ],
27     "id": "3"
28   },
29   {
30     "links": [
31       {
32         "rel": "self",
33         "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/currency/4"
34       }
35     ],
36     "id": "4"
37   },
38   {
39     "links": [
40       {
41         "rel": "self",
42         "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/currency/1"
43       }
44     ],
45     "id": "1"
46   },
47   {
48     "links": [
49       {
50         "rel": "self",
51         "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/currency/5"
52       }
53     ],
54     "id": "5"
55   }
56 ],
57 "offset": 0,
58 "totalResults": 5
59 }

```

getAsyncResult

In SOAP web services, you use the `getAsyncResult` operation to retrieve details about specific records submitted as part of an asynchronous request.

SOAP Request

You can use code like the following to retrieve the `getAsyncResult` response. This example shows how to look for records that failed with a status detail code of "unexpected error" and then take some action with those records.

```

1 AsyncAddListResult result = (AsyncAddListResult)port.getAsyncResult("job ID", 1);
2 WriteResponseList responses = result.getWriteResponseList();
3 for (WriteResponse response : responses.getWriteResponse())
4 {
5     if (!response.getStatus().isSuccess())
6     {
7         if (response.getStatus().getStatusDetail()[0].getCode() == StatusDetailCodeType.UNEXPECTED_ERROR)
8             // add for resubmit
9         }
10 }

```

REST Request

In REST web services, you can use the async service to retrieve details about a single async operation. For detailed information about asynchronous processing in REST and for more examples, see the help topic [REST Web Services Request Processing](#).

You can send a request similar to the following to get the result of the executed request: GET <https://demo123.suitetalk.api.netsuite.com/services/rest/async/v1/job/1/task/1/result>.

The following example shows the returned result:

```

1 {
2   "links": [
3     {
4       "rel": "self",
5       "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer/... "
6     }
7   ],
8   "companyName": "Glenrock General Hospital",
9   "email": "alan.smith@example.com",
10  "entityStatus": {
11    "links": [
12      {
13        "rel": "self",
14        "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customerstatus/13"
15      }
16    ],
17    "id": "13",
18    "refName": "CUSTOMER-Closed Won"
19  }
20 }
```

getCurrencyRate

In SOAP web services, you use the `getCurrencyRate` operation to get the exchange rate between two currencies based on a certain date. The exchange rate values are those that appear in the Exchange Rate column of the Currency Exchange Rates table, which you can view at Lists > Accounting > Currency Exchange Rates.

The role used to perform this operation must have at least View level of the Currency permission (a Lists type permission).

The Multiple Currencies feature must be enabled in your account before using the `getCurrencyRate` operation.

SOAP Request

The following example shows the usage of the `getCurrencyRate` operation.

```

1 <soapenv:Envelope xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:soapenv
2   v="http://schemas.xmlsoap.org/soap/envelope/">
3   <soapenv:Header>
4     <ns1:passport soapenv:mustUnderstand="0" soapenv:actor="http://schemas.xmlsoap.org/soap/actor/next" xmlns:ns1="urn:mes
5     sages_2025_1.platform.webservices.netsuite.com">
6       <ns2:email xmlns:ns2="urn:core_2025_1.platform.webservices.netsuite.com">email@netsuite.com</ns2:email>
7       <ns3:password xmlns:ns3="urn:core_2025_1.platform.webservices.netsuite.com">*****</ns3:password>
8       <ns4:account xmlns:ns4="urn:core_2025_1.platform.webservices.netsuite.com">1234567</ns4:account>
9       <ns5:role internalId="37" xmlns:ns5="urn:core_2025_1.platform.webservices.netsuite.com"/>
10    </ns1:passport>
11  </soapenv:Header>
```

```

10 <soapenv:Body>
11   <getCurrencyRate xmlns="urn:messages_2025_1.platform.webservices.netsuite.com">
12     <currencyRateFilter>
13       <ns6:baseCurrency internalId="2" xmlns:ns6="urn:core_2025_1.platform.webservices.netsuite.com"/>
14       <ns7:fromCurrency internalId="1" xmlns:ns7="urn:core_2025_1.platform.webservices.netsuite.com">
15       <ns8:effectiveDate xmlns:ns8="urn:core_2025_1.platform.webservices.netsuite.com">2012-08
16         11T06:58:59.628Z</ns8:effectiveDate>
17     </currencyRateFilter>
18   </getCurrencyRate>
19 </soapenv:Body>
20 </soapenv:Envelope>

```

SOAP Response

The following sample shows the response to a getCurrencyRate request.

```

1 <soapenv:Envelope xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:soapen
  v="http://schemas.xmlsoap.org/soap/envelope/">
2   <soapenv:Header>
3     <platformMsgs:documentInfo xmlns:platformMsgs="urn:messages_2025_1.platform.webservices.netsuite.com">
4       <platformMsgs:nsId>WEBSERVICES_1326288_08092012257172984776216537_6c7c330232bb5</platformMsgs:nsId>
5     </platformMsgs:documentInfo>
6   </soapenv:Header>
7   <soapenv:Body>
8     <getCurrencyRateResponse xmlns="urn:messages_2025_1.platform.webservices.netsuite.com">
9       <platformCore:getCurrencyRateResult xmlns:platformCore="urn:core_2025_1.platform.webservices.netsuite.com">
10        <platformCore:status isSuccess="true"/>
11        <platformCore:currencyRateList>
12          <platformCore:currencyRate>
13            <platformCore:baseCurrency internalId="2"/>
14            <platformCore:fromCurrency internalId="1"/>
15            <platformCore:exchangeRate>0.42283298</platformCore:exchangeRate>
16            <platformCore:effectiveDate>1970-01-01T00:00:00.000-08:00</platformCore:effectiveDate>
17          </platformCore:currencyRate>
18        </platformCore:currencyRateList>
19      </platformCore:getCurrencyRateResult>
20    </getCurrencyRateResponse>
21  </soapenv:Body>
22 </soapenv:Envelope>

```

REST Request

In REST web services, you can work with currency exchange rates by sending a GET request for a specific currency exchange rate record.

You must enable the Multiple Currencies feature before you can use this record through REST web services.

The following example retrieves a specific currency exchange rate record.

```

1 | GET https://test.suitetalk.api.netsuite.com/services/rest/record/v1/currencyrate/20208

```

REST Response

The following example shows the result of the GET operation.

```

1 {
2   "links": [
3     {
4       "rel": "self",

```

```

5      "href": "https://test.suitetalk.api.netsuite.com/services/rest/record/v1/currencyrate/20208"
6    }
7  ],
8  "baseCurrency": {
9    "links": [
10     {
11       "rel": "self",
12       "href": "https://test.suitetalk.api.netsuite.com/services/rest/record/v1/currency/1"
13     }
14   ],
15   "id": "1",
16   "refName": "USA"
17 },
18 "effectiveDate": "2025-07-22",
19 "exchangeRate": 1.34956,
20 "fxSourceMethod": "DIRECT",
21 "id": "20208",
22 "lastModifiedDate": "2025-07-22T10:16:00Z",
23 "transactionCurrency": {
24   "links": [
25     {
26       "rel": "self",
27       "href": "https://test.suitetalk.api.netsuite.com/services/rest/record/v1/currency/2"
28     }
29   ],
30   "id": "2",
31   "refName": "British pound"
32 }
33 }

```

getIntegrationGovernanceInfo

In SOAP web services, the `getIntegrationGovernanceInfo` operation can be used to retrieve the available concurrency limit for the specific integration.

The operation does not have any parameters and returns the value for concurrency limit and the limit type. The limit type can be:

- `_integrationSpecific`, when a limit has been allocated to the integration.
- `_accountLimit`, when no specific limit has been allocated to the integration.
- `_internal`, when it is an internal application. In this case the limit is not displayed.

SOAP Request

The following request shows the usage of the operation.

```

1 <soapenv:Body>
2   <getIntegrationGovernanceInfo />
3 </soapenv:Body>

```

SOAP Response

The following response shows the integration specific concurrency limit.

```

1 <soapenv:Body>
2   <getIntegrationGovernanceInfoResponse>
3     <platformCore:getIntegrationGovernanceInfoResult>
4       <platformCore:status isSuccess="true"/>
5       <platformCore:integrationConcurrencyLimit>2</platformCore:integrationConcurrencyLimit>

```

```

6         <platformCore:limitType>_integrationSpecific</platformCore:limitType>
          </platformCore:getIntegrationGovernanceInfoResult>
7     </getIntegrationGovernanceInfoResponse>
8 </soapenv:Body>

```

REST Request

In REST web services, you use the `governanceLimits` operation to retrieve information about concurrency limits.

The `governanceLimits` operation does not have any parameters and returns (1) the account concurrency limit, which specifies the maximum number of concurrent connections allowed for a given account, (2) the unallocated concurrency limit, which indicates a limit for all integrations without a specifically defined limit, and (3) the limit type.

The following example shows the usage of the `governanceLimits` operation.

```

1 | GET https://demo123.suitetalk.api.netsuite.com/services/rest/system/v1/governanceLimits

```

REST Response

The following sample shows the integration specific concurrency limit.

```

1 {
2   "accountConcurrencyLimit": 5,
3   "accountUnallocatedConcurrencyLimit": 3,
4   "integrationConcurrencyLimit": 2,
5   "integrationLimitType": "integrationSpecific"
6 }

```

getItemAvailability

In SOAP web services, the `getItemAvailability` operation is used to retrieve the inventory availability for a specific list of items.

If the Multi-Location Inventory feature is enabled, this operation returns results for all locations. For locations that do not have any items available, only location IDs and names are listed in results.

SOAP Request

The following request shows the usage of the `getItemAvailability` operation.

```

1 <soap:Body>
2   <platformMsgs:getItemAvailability>
3     <platformMsgs:itemAvailabilityFilter>
4       <platformCore:item>
5         <platformCore:recordRef internalId="390" type="inventoryItem">
6           <platformCore:name/>
7         </platformCore:recordRef>
8       </platformCore:item>
9       <platformCore:lastQtyAvailableChange/>
10    </platformMsgs:itemAvailabilityFilter>
11  </platformMsgs:getItemAvailability>
12 </soap:Body>

```

SOAP Response

The following sample shows the response to the previous request.

```

1 <soapenv:Body>
2 <getItemAvailabilityResponse xmlns="urn:messages_2025_1.platform.webservices.netsuite.com">
3   <getItemAvailabilityResult xmlns="urn:core_2025_1.platform.webservices.netsuite.com">
4     <status isSuccess="true"/>
5     <itemAvailabilityList>
6       <itemAvailability>
7         <item internalId="390" type="inventoryItem">
8           <name>testItem</name>
9         </item>
10        <locationId internalId="1" type="location">
11          <name>East Coast</name>
12        </locationId>
13        <quantityOnHand>20.0</quantityOnHand>
14        <onHandValueMli>0.0</onHandValueMli>
15        <quantityCommitted>0.0</quantityCommitted>
16        <quantityAvailable>20.0</quantityAvailable>
17      </itemAvailability>
18      <itemAvailability>
19        <item internalId="390" type="inventoryItem">
20          <name>testItem</name>
21        </item>
22        <locationId internalId="2" type="location">
23          <name>West Coast</name>
24        </locationId>
25      </itemAvailability>
26    </itemAvailabilityList>
27  </getItemAvailabilityResult>
28 </getItemAvailabilityResponse>
29 </soapenv:Body>

```

REST Request

In REST web services, you can get information about item availability by sending a GET request to retrieve a specific item.

```
1 | GET https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/inventoryItem/705?expandSubResources=true
```

REST Response

The following sample shows a shortened response.

```

1 {
2   "links": [
3     {
4       "rel": "self",
5       "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/inventoryItem/705"
6     }
7   ],
8   "id": "705",
9   "internalId": 705,
10  "itemId": "Sample Boxes",
11  "lastModifiedDate": "2024-04-25T15:06:00Z",
12  "lastPurchasePrice": 0.0,
13  "manufacturerState": "CA",
14  "matchBillToReceipt": false,
15  "multManufactureAddr": false,
16  "nextTagProductFeed": false,
17  "offerSupport": false,

```

```

18  "onHandValueMli": "30",
19  "quantityAvailable": "40",
20  "quantityCommitted": "0",
21  "quantityOnHand": "20",
22  "roundUpAsComponent": false,
23  "saleUnit": "9",
24  "seasonalDemand": false,
25  "shipIndividually": false,
26  "shoppingProductFeed": false,
27  "shopzillaProductFeed": false,
28  "stockUnit": "9",
29  "totalValue": 0.0,
30  "trackLandedCost": false,
31  "weightUnits": "per Smp bx",
32  "yahooProductFeed": false
33  }

```

getList

In SOAP web services, the `getList` operation is used to retrieve a list of one or more records by providing the unique ids that identify those records.

If there are multiple ids provided, they can either belong to the same record type or different record types. For example, it is possible to retrieve a customer and a contact within a single request using this operation.

SOAP Request

In the following example, two records are retrieved — one customer record and one employee record. Note that you must provide the internal ID of the specify instance of the record and the record type for the `getList`.

```

1  <soap:Body>
2  <platformMsgs:getList>
3    <platformMsgs:baseRef internalId="983" type="customer" xsi:type="platformCore:RecordRef"/>
4    <platformMsgs:baseRef internalId="-5" type="employee" xsi:type="platformCore:RecordRef"/>
5  </platformMsgs:getList>
6  </soap:Body>

```

SOAP Response

The following example shows the response to the previous SOAP request.

```

1  <soapenv:Body>
2  <getListResponse xmlns="urn:messages_2025_1.platform.webservices.netsuite.com">
3  <readResponseList xmlns="urn:messages_2025_1.platform.webservices.netsuite.com">
4    <readResponse>
5      <ns1:status isSuccess="true" xmlns:ns1="urn:core_2025_1.platform.webservices.netsuite.com"/>
6      <record internalId="983" xsi:type="ns2:Customer" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:ns2="urn:relationships_2025_1.lists.webservices.netsuite.com">
7        <ns2:entityId>Shutter Fly</ns2:entityId>
8        <ns2:isInactive>false</ns2:isInactive>
9        <ns2:companyName>Shutter Fly, Inc</ns2:companyName>
10       .
11       ...[more fields]
12       .
13       <ns2:customFieldList>
14         <ns6:customField internalId="265" scriptId="custentity_map" xsi:type="ns6:StringCustomFieldRef" xmlns:ns6="urn:core_2025_1.platform.webservices.netsuite.com">
15           <ns6:value>http://maps.google.com</ns6:value>

```

```

16         </ns6:customField>
17     </ns2:customFieldList>
18 </record>
19 </readResponse>
20 <readResponse>
21     <ns8:status isSuccess="true" xmlns:ns8="urn:core_2025_1.platform.webservices.netsuite.com"/>
22     <record internalId="-5" xsi:type="ns9:Employee" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:ns9="urn:employ
ees_2025_1.lists.webservices.netsuite.com">
23         <ns9:entityId>A Wolfe</ns9:entityId>
24         <ns9:isInactive>false</ns9:isInactive>
25         .
26         ...[more fields]
27         .
28     </record>
29 </readResponse>
30 </readResponseList>
31 </getListResponse>
32 </soapenv:Body>

```

REST Request

In REST web services, you can obtain the list of all records of a record type by sending an HTTP GET request as in the following example.

```
1 | GET https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer
```

REST Response

The following is a shortened example of a response.

```

1  {
2    "links": [
3      {
4        "rel": "self",
5        "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer?limit=1000&offset=0"
6      }
7    ],
8    "items": [
9      {
10       "links": [
11         {
12           "rel": "self",
13           "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer/107"
14         }
15       ],
16       "id": "107"
17     },
18     {
19       "links": [
20         {
21           "rel": "self",
22           "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer/41"
23         }
24       ],
25       "id": "41"
26     },
27     {
28       "links": [
29         {
30           "rel": "self",
31           "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer/90"
32         }
33       ],
34       "id": "90"
35     }
36   ]
37 }

```



```

36 | ],
37 |   "totalResults": 3
38 | }

```

getServerTime

The `getServerTime` operation in SOAP web services takes no arguments and returns the NetSuite server time in GMT, regardless of a user's time zone. Developers do not have to rely on client time when writing synchronization procedures because the client time may not be in synch with the NetSuite server time.

SOAP Request

The following Java code sample shows how to send a `getServerTime` request.

```

1 | public void testGetServerTime() throws Exception
2 | {
3 |     c.login();
4 |
5 |     GetServerTimeResult rs = c.getPort().getServerTime();
6 |     System.out.println("Welcome to NetSuite. At the sound of the tone the NetSuite time will be : " + new
7 |     SimpleDateFormat("yyyy-MM-dd HH:mm:ss").format(rs.getServerTime().getTime()));
8 |
9 |     System.out.println("This compares with a client time of " + new SimpleDateFormat("yyyy-MM-dd
10 |    HH:mm:ss").format(Calendar.getInstance().getTime() ));
11 |     System.out.println("This represents a skew of " + (Calendar.getInstance().getTimeInMillis()
12 |     rs.getServerTime().getTimeInMillis() )/1000 + " seconds from netsuite
13 |     (where positive time means the client clock is comparatively fast).");
14 | }

```

REST Request

In REST web services, the `serverTime` operation returns the NetSuite server time in UTC, regardless of a user's time zone.

The following example shows how to send a `serverTime` request.

```

1 | GET https://demo123.suitetalk.api.netsuite.com/services/rest/system/v1/serverTime

```

REST Response

The following example shows that the `dateTime` value is returned by `serverTime` is in UTC time zone.

```

1 | {
2 |   "serverTime": "2025-03-26T16:21:00.000Z"
3 | }

```

initialize / initializeList

You can use the `initialize` operation in SOAP web services to emulate the UI workflow by prepopulating fields on transaction line items with values from a related record. Your SOAP web services application can then modify only the values it needs to before submitting the record.

SOAP Request

The following example shows the usage of the initialize operation.

```

1 <soapenv:Body>
2 <platformMsgs:initialize xmlns:soapenc="http://schemas.xmlsoap.org/soap/encoding/" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:xs="http://www.w3.org/2001/XMLSchema" xmlns:platformCoreType="urn:types.core_2025_1.platform.webservices.netsuite.com" xmlns:platformCore="urn:core_2025_1.platform.webservices.netsuite.com" xmlns:platformMsgs="urn:messages_2025_1.platform.webservices.netsuite.com">
3   <platformMsgs:initializeRecord>
4     <platformCore:type>invoice</platformCore:type>
5     <platformCore:reference internalId="1513" type="salesOrder">
6       <platformCore:name>1511</platformCore:name>
7     </platformCore:reference>
8   </platformMsgs:initializeRecord>
9 </platformMsgs:initialize>
10 </soapenv:Body>

```

The following example shows the usage of the initializeList operation.

```

1 <initializeList xmlns="urn:messages_2025_1.platform.webservices.netsuite.com">
2   <initializeRecord>
3     <ns1:type xmlns:ns1="urn:core_2025_1.platform.webservices.netsuite.com">customerPayment
4   </ns1:type>
5     <ns2:reference internalId="176" type="customer" xmlns:ns2="urn:core_2025_1.platform.webservices.netsuite.com"/>
6   </initializeRecord>
7   <initializeRecord>
8     <ns3:type xmlns:ns3="urn:core_2025_1.platform.webservices.netsuite.com">
9       invoice</ns3:type>
10     <ns4:reference internalId="176" type="customer" xmlns:ns4="urn:core_2025_1.platform.webservices.netsuite.com"/>
11   </initializeRecord>
12 </initializeList>

```

REST Request

In REST web services, you can transform a record from one type into another, using data from an existing record. This is the equivalent of the initialize SOAP operation. For example, you can create an invoice record from an existing sales order record, using data from the sales order.

In REST web services, you use the POST method to send a record transformation request. The record transformation is executed in a single request. In the request URL, the following details must be specified:

- The record type you that you want to transform.
- The ID of the record that you want to transform.
- The transformation action. The transformation action must be formatted in the following way in the URL: !transform.
- The record type you that you want transform into.

In the following example, a sales order record is transformed into an invoice. If the transformation is performed successfully, an HTTP 204 – No Content response is returned.

```
1 | POST http://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/salesOrder/3!/transform/invoice
```

In the request body, you can also specify data for additional fields of the record, as shown in the following example.

```

1 {
2   "memo": "REST, end date and bills date added",
3   "enddate": "2020-01-14",

```

```

4      "billdate": "2020-01-14",
5      "item": {
6          "items": [
7              {
8                  "item": { "id": "3456" },
9                  "amount": 1
10             }
11         ]
12     }
13 }

```

search

In SOAP web services, the search operation is used to run a search on a specific record type based on a set of criteria. You can search by defining search filter fields on the record, joined fields on an associated record, search return columns, or joined search return columns from an associated record. The results of the search can be complete records, or a select set of fields specified through search return columns.

SOAP Request

The following example shows a search for customer records that have an email that contains example.com.

```

1 <soap:Body>
2 <platformMsgs:search>
3 <searchRecord xsi:type="ContactSearch">
4   <customerJoin xsi:type="CustomerSearchBasic">
5     <email operator="contains" xsi:type="platformCore:SearchStringField">
6       <platformCore:searchValue>example.com</platformCore:searchValue>
7     </email>
8   </customerJoin>
9 </searchRecord>
10 </search>
11 </soap:Body>

```

SOAP Response

The following example shows the result of the previous request.

```

1 <soapenv:Body>
2 <searchResponse xmlns="urn:messages_2025_1.platform.webservices.netsuite.com">
3 <searchResult xmlns="urn:core_2025_1.platform.webservices.netsuite.com">
4 <status isSuccess="true"/>
5 <totalRecords>1</totalRecords>
6 <pageSize>10</pageSize>
7 <totalPages>1</totalPages>
8 <pageIndex>1</pageIndex>
9 <recordList>
10 <record internalId="983" xsi:type="ns1:Customer" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:ns1="urn:relation
ships_2025_1.lists.webservices.netsuite.com">
11 <ns1:entityId>Shutter Fly</ns1:entityId>
12 <ns1:isInactive>false</ns1:isInactive>
13 <ns1:companyName>Shutter Fly, Inc</ns1:companyName>
14 <ns1:entityStatus internalId="6"><name>LEAD-New</name>
15 .
16 .
17 .
18 <ns1:customFieldList>
19 <customField internalId="76" scriptId="custentity_had_order_problems" xsi:type="BooleanCustomFieldRef"><value>false</
value></customField>

```

```

20 </ns1:customFieldList>
21 </record>
22 </recordList>
23 </searchResult>
24 </searchResponse>
25 </soapenv:Body>

```

REST Request

In REST web services, you can filter the collection of all record instances by using the `q` query parameter to specify filter conditions. Each condition consists of a field name, an operator, and a value. You can join several conditions using the AND / OR logical operators, and you can use `()` to mark precedence. For more information, see the help topic [Record Collection Filtering](#).

The following is an example of a simple query. The spaces in URLs are encoded. The following examples are presented without encoding for clarity.

```
1 GET https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer?q=email START_WITH barbara
```

REST Response

The response is a collection of customer record instances where the value in the email field starts with the value of barbara. The result is a collection resource containing links to resources that match query criteria. The response could be similar to the following.

```

1 {
2   "links": [
3     {
4       "rel": "self",
5       "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer?limit=1000&offset=0"
6     }
7   ],
8   "items": [
9     {
10      "links": [
11        {
12          "rel": "self",
13          "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer/107"
14        }
15      ],
16      "id": "107"
17    },
18  ],
19  "totalResults": 1
20 }

```

Besides record collection filtering, you can also execute SuiteQL queries through REST web services by sending a POST request to the `suiteql` resource, and specifying the query in the request body after the body parameter `q`. Using SuiteQL queries, you can return a maximum of 100,000 results. If you expect your queries to return more than 100,000 results, you can use SuiteAnalytics Connect with the NetSuite2.com data source. For more information, see the help topic [Executing SuiteQL Queries Through REST Web Services](#).

The following example shows a SuiteQL query executed through REST web services. Note that `Prefer: transient` is a required header parameter.

```

1 POST https://demo123.suitetalk.api.netsuite.com/services/rest/query/v1/suiteql?limit=10&offset=10
2 Prefer: transient
3 {
4   "q": "SELECT email, COUNT(*) as count FROM transaction GROUP BY email"

```

```
5 | }
```

The following is a shortened response.

```
1 {
2   "links": ...,
3   "count": 3,
4   "offset": 10,
5   "totalResults": 53,
6   "items": [
7     {
8       "links": [],
9       "email": "test@netsuite.com",
10      "count": "143"
11    },
12    {
13      "links": [],
14      "email": "test2@netsuite.com",
15      "count": "144"
16    },
17    {
18      "links": [],
19      "email": "test3@netsuite.com",
20      "count": "145"
21    }
22  ],
23  ...
24 }
```

searchMoreWithId

In SOAP web services, the searchMoreWithId operation enables you to reference a specific search result set by its searchId, a parameter included in all search results.

When you paginate through the search results, the searchMoreWithId operation triggers a new search for the page specified by the pageIndex value.

SOAP Samples

The following samples show the SOAP request for the initial search as well as the SOAP request for a subsequent searchMoreWithId operation used to specify the next page of the search. The ID returned in the initial search is specified as the searchId when executing searchMoreWithId.

The first sample shows the initial search.

```
1 <soapenv:Envelope>
2   <soapenv:Header>
3     <platformMsgs:searchPreferences>
4       <platformMsgs:bodyFieldsOnly>true</platformMsgs:bodyFieldsOnly>
5       <platformMsgs:pageSize>100</platformMsgs:pageSize>
6     </platformMsgs:searchPreferences>
7     <platformMsgs:passport>
8       <platformCore:email>jdoe@netsuite.com</platformCore:email>
9       <platformCore:password><SOAP web services password></platformCore:password>
10      <platformCore:account>000034</platformCore:account>
11      <platformCore:role internalId="37"/>
12    </platformMsgs:passport>
13   </soapenv:Header>
14   <soapenv:Body>
15     <platformMsgs:search>
16       <searchRecord xsi:type="platformCommon:ContactSearchBasic">
17         <platformCommon:city operator="is" xsi:type="platformCore:SearchStringField">
18           <platformCore:searchValue xsi:type="xsd:string">San Francisco</platformCore:searchValue>
```

```

19         </platformCommon:city>
20     </searchRecord>
21 </platformMsgs:search>
22 </soapenv:Body>
23 </soapenv:Envelope>

```

The following sample shows the result of the initial search. Note the searchId in the response.

```

1 <soapenv:Envelope>
2   <soapenv:Header>
3     <platformMsgs:documentInfo>
4       <platformMsgs:nsId>WEBSERVICES_528736_07012008543995006307049233_d53ef4d2273b15</platformMsgs:nsId>
5     </platformMsgs:documentInfo>
6   </soapenv:Header>
7   <soapenv:Body>
8     <platformMsgs:searchResponse>
9       <platformCore:searchResult>
10        <platformCore:status isSuccess="true"/>
11        <platformCore:totalRecords>231</platformCore:totalRecords>
12        <platformCore:pageSize>100</platformCore:pageSize>
13        <platformCore:totalPages>3</platformCore:totalPages>
14        <platformCore:pageIndex>1</platformCore:pageIndex>
15        <platformCore:searchId> WEBSERVICES_528736_07012008543995006307049233_d53ef4d2273b15
16      </platformCore:searchId>
17      <platformCore:recordList>
18        <platformCore:recordList>
19          <platformCore:record internalId="4" externalId="entity-4" xsi:type="listRel:Contact">
20            <listRel:entityId>john</listRel:entityId>
21            <listRel:firstName>John</listRel:firstName>
22            .....
23          </platformCore:record>
24        </platformCore:recordList>
25      </platformCore:searchResult>
26    </searchMoreWithIdResponse>
27  </soapenv:Body>
28 </soapenv:Envelope>

```

In the following sample, you use the searchId to retrieve the next page of results using the searchMoreWithId operation.

```

1 <soapenv:Envelope>
2   <soapenv:Header>
3     <platformMsgs:searchPreferences>
4       <platformMsgs:bodyFieldsOnly>true</platformMsgs:bodyFieldsOnly>
5       <platformMsgs:pageSize>100</platformMsgs:pageSize>
6     </platformMsgs:searchPreferences>
7     <platformMsgs:passport>
8       <platformCore:email>jdoe@netsuite.com</platformCore:email>
9       <platformCore:password><SOAP web services password></platformCore:password>
10      <platformCore:account>000034</platformCore:account>
11      <platformCore:role internalId="37"/>
12    </platformMsgs:passport>
13  </soapenv:Header>
14  <soapenv:Body>
15    <platformMsgs:searchMoreWithId>
16      <searchId>WEBSERVICES_528736_07012008543995006307049233_d53ef4d2273b15</searchId>
17      <pageIndex>2</pageIndex>
18    </platformMsgs:searchMoreWithId>
19  </soapenv:Body>
20 </soapenv:Envelope>

```

The following example shows the result of the searchMoreWithId operation. The next page of results is returned.

```

1 <soapenv:Envelope>
2   <soapenv:Header>
3     <platformMsgs:documentInfo>
4       <platformMsgs:nsId>WEBSERVICES_528736_07012008543995006307049233_d53ef4d2273b15</platformMsgs:nsId>

```

```

5      </platformMsgs:documentInfo>
6    </soapenv:Header>
7    <soapenv:Body>
8      <platformMsgs:searchMoreWithIdResponse>
9        <platformCore:searchResult>
10         <platformCore:status isSuccess="true"/>
11         <platformCore:totalRecords>231</platformCore:totalRecords>
12         <platformCore:pageSize>100</platformCore:pageSize>
13         <platformCore:totalPages>3</platformCore:totalPages>
14         <platformCore:pageIndex>1</platformCore:pageIndex>
15         <platformCore:searchId > WEBSERVICES_528736_07012008543995006307049233_d53ef4d2273b15
16         </platformCore:searchId >
17         <platformCore:recordList>
18           <platformCore:record internalId="5" externalId="entity-5" xsi:type="listRel:Contact">
19             <listRel:entityId>mike</listRel:entityId>
20             <listRel:firstName>Mike</listRel:firstName>
21             ....
22           </platformCore:record>
23         </platformCore:recordList>
24       </platformCore:searchResult>
25     </platformMsgs:searchMoreWithIdResponse>
26   </soapenv:Body>
27 </soapenv:Envelope>

```

REST Samples

In REST web services, lists of record instances and the results of saved analytics workbook searches are returned in one or more pages. The results are displayed on multiple pages, with the default setting of 1000 results per page on a maximum of 1000 pages. This way you can return maximum one million results if you use SuiteAnalytics Connect. Without SuiteAnalytics Connect, you can display maximum 100 000 results.

You can also specify your own paging by adding the limit criteria to your request.

The following examples show how to move between pages of returned results.

First, you send a request for all instance of customer records, with two results displayed per page.

```
1 | GET https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer?limit=2
```

The following example shows the response.

```

1 {
2   "links": [
3     {
4       "rel": "next",
5       "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/account?limit=2&offset=2"
6     },
7     {
8       "rel": "last",
9       "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/account?limit=22&offset=2"
10    },
11    {
12      "rel": "self",
13      "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer?limit=2&offset=0"
14    }
15  ],
16  "items": [
17    {
18      "links": [
19        {
20          "rel": "self",
21          "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer/107"
22        }
23      ],
24      "id": "107"

```

```

25     },
26     {
27         "links": [
28             {
29                 "rel": "self",
30                 "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer/41"
31             }
32         ],
33         "id": "41"
34     }
35 ],
36 "count": 2,
37 "offset": 0,
38 "hasMore": true,
39 "totalResults": 3
40 }

```

The values of the count, hasMore, and totalResults fields provide information about the results that you can retrieve in subsequent requests for other pages. Links to the last and next pages provide direct links that you can use to retrieve these pages.

You can retrieve the next page by sending a request with the offset value specified. The offset provides the index of the result from which you should start the requested page.

The following is an example of a request for the second (and last) page of results with all customers.

```
1 | GET https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/account?limit=2&offset=2
```

The following example shows the result of the request.

```

1  {
2      "links": [
3          {
4              "rel": "prev",
5              "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer?limit=2&offset=0"
6          },
7          {
8              "rel": "first",
9              "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer?limit=2&offset=0"
10         },
11         {
12             "rel": "self",
13             "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer?limit=2&offset=2"
14         }
15     ],
16     "items": [
17         {
18             "links": [
19                 {
20                     "rel": "self",
21                     "href": "https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer/90"
22                 }
23             ],
24             "id": "90"
25         }
26     ],
27     "count": 1,
28     "offset": 2,
29     "hasMore": false,
30     "totalResults": 3
31 }

```

update

In SOAP web services, the update operation is used to update an instance of a record.

Only the fields that have been populated in each submitted record are updated. If a field has NOT been populated, it is not updated and it retains its previous value. If a field is set to an empty string, the previous value of the field is replaced with an empty string.

SOAP Request

In the following example, a customer's `companyName` is updated. The internal ID for the customer must be provided in the request.

```

1 <soap:Body>
2 <platformMsgs:update>
3   <platformMsgs:record internalId="980" xsi:type="listRel:Customer">
4     <listRel:companyName>Shutter Fly Corporation</listRel:companyName>
5   </platformMsgs:record>
6 </platformMsgs:update>
7 </soap:Body>

```

SOAP Response

The following example shows that the previous request completed successfully.

```

1 <soapenv:Body>
2 <updateResponse xmlns="urn:messages_2025_1.platform.webservices.netsuite.com">
3   <writeResponse xmlns="urn:messages_2025_1.platform.webservices.netsuite.com">
4     <ns1:status isSuccess="true" xmlns:ns1="urn:core_2025_1.platform.webservices.netsuite.com"/>
5     <baseRef internalId="980" type="customer" xsi:type="ns2:RecordRef" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
6       instance" xmlns:ns2="urn:core_2025_1.platform.webservices.netsuite.com"/>
7   </writeResponse>
8 </updateResponse>
9 </soapenv:Body>

```

REST Request

In REST web services, you can update a record instance using the PATCH HTTP method. The PATCH method expects a request body with the same fields that can be retrieved using the GET method. That is, the record's metadata schema is shared between reads and updates.

In the following example, the `companyName` of a specific customer record instance is updated. In this example, the company name of the record instance 980 is changed to Shutter Fly Corporation. Send the following request to perform the update.

```

1 PATCH https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer/980
2 BODY { "companyName": "Shutter Fly Corporation" }

```

REST Response

The following is an excerpt from the response headers of a successful operation, with no body content returned (HTTP Code 204). The URL of the updated record is given in the Location header of the response.

```

1 Date Fri, 04 Jan 2025 09:07:01 GMT
2 Location https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer/980
3 Content-Type application/json

```

updateInviteeStatus

In SOAP web services, the updateInviteeStatus operation lets users respond to NetSuite events that have been sent to them. This operation takes both the internal ID of the event as well as a calendar event status as arguments.

After invitees have responded to the event invitation, the event record is updated with their response. Possible responses include accepted, declined, tentative, and no response.

SOAP Request

The following example shows the usage of the updateInviteeStatus operation.

```
1 <updateInviteeStatus xmlns="urn:messages_2025_1.platform.webservices.netsuite.com">
2   <updateInviteeStatusReference>
3     <eventId internalId="100" xmlns="urn:core_2025_1.platform.webservices.netsuite.com" />
4     <responseCode xmlns="urn:core_2025_1.platform.webservices.netsuite.com">_accepted
5   </responseCode>
6   </updateInviteeStatusReference>
7 </updateInviteeStatus>
```

SOAP Response

The following example shows that the previous request was completed successfully.

```
1 <updateInviteeStatusResponse xmlns="urn:messages_2025_1.platform.webservices.netsuite.com">
2   <writeResponse>
3     <platformCore:status isSuccess="true" xmlns:platformCore="urn:core_2025_1.platform.webservices.netsuite.com"/>
4     <baseRef internalId="100" type="calendarEvent" xsi:type="platformCore:RecordRef" xmlns:platformCore="urn:core_2025_1.plat
5   form.webservices.netsuite.com"/>
6   </writeResponse>
7 </updateInviteeStatusResponse>
```

REST Request

In REST web services, you can update the invitee status of events by sending a PATCH request to the specific calendar event record. The request body should contain the updated status.

```
1 PATCH https://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/calendarEvent/1857
2
3 Request body:
4
5 {
6   "id": "ACCEPTED",
7   "refName": "Accepted"
8 }
```

If the request is successful, an HTTP 204 No Content response is returned.

upsert

In SOAP web services, the upsert operation is used to add a new instance or to update an instance of a record in NetSuite.

The upsert operation is similar to both the add and update operations, but upsert can be run without first determining whether a record exists in NetSuite. A record is identified by its external ID and its record type. If a record of the specified type with a matching external ID exists in the system, it is updated. If it does not exist, a new record is created.

Because external ID is required for this operation, upsert is supported only for records that support the external ID field.

SOAP Request

The following example shows how to use the upsert operation.

```

1 <soap:Body>
2   <upsert xmlns="urn:messages_2025_1.platform.webservices.netsuite.com">
3     <record xmlns:q1="urn:relationships_2025_1.lists.webservices.netsuite.com" xsi:type="q1:Customer" externalId="THISISMYEXTID">
4       <q1:entityId>XYZ 2 Inc</q1:entityId>
5       <q1:companyName>XYZ 2, Inc.</q1:companyName>
6       <q1:email>bsanders@xyz.com</q1:email>
7     </record>
8   </upsert>
9 </soap:Body>

```

SOAP Response

The following example shows the result of a successful operation.

```

1 <soapenv:Body>
2   <upsertResponse xmlns="urn:messages_2025_1.platform.webservices.netsuite.com">
3     <writeResponse>
4       <platformCore:status isSuccess="true" xmlns:platformCore="urn:core_2025_1.platform.webservices.netsuite.com"/>
5       <baseRef internalId="973" externalId="THISISMYEXTID" type="customer" xsi:type="platformCore:RecordRef" xmlns:platformCore="urn:core_2025_1.platform.webservices.netsuite.com"/>
6     </writeResponse>
7   </upsertResponse>
8 </soapenv:Body>

```

REST Request

In REST web services, you can use the PUT operation with an external ID as a key to a record instead of an internal ID to create or update a record instance.

In REST web services, an external ID starts with the prefix "eid:" in the following format: eid:external_id.

You can use external IDs anywhere in the URL where an internal ID can be used.

The following example shows how to add or update a customer record.

```

1 PUT http://demo123.suitetalk.api.netsuite.com/services/rest/record/v1/customer/eid:CID001
2 BODY {
3   "firstName": "John",
4   "lastName": "Smith",
5   "isPerson": true,
6   "subsidiary": {
7     "id": "1"
8   }
9 }

```

If the request is successful, an HTTP 204 No Content response is returned.

SOAP Web Services and REST Web Services Supported Records

The following table provides a comparison of the record types supported for SOAP web services and REST Web Services.

 **Tip:** For information about supported records in every integration channel, see the help topic [SuiteCloud Supported Records](#).

Record Category	Record Type	SOAP Web Services	REST Web Services
Activities	Event	✓	✓
	Phone Call	✓	✓
	Project Task	✓	✓
	Resource Allocation	✓	—
	Task	✓	✓
Communications	Message	✓	✓
	Note	✓	✓
	Note Type	✓	✓
Customization	Custom Field (values)	✓	You can use CRUD operations on custom field values in the context of records. For details, see the help topic Using CRUD Operations on Custom Records, Custom Lists, and Custom Transactions v1 .
	Custom List	✓	You can use CRUD operations on custom lists. For details, see the help topic Using CRUD Operations on Custom Records, Custom Lists, and Custom Transactions v1 .
	Custom Record (instances)	✓	You can use CRUD operations on custom records. For details, see the help topic Using CRUD Operations on Custom Records, Custom Lists, and Custom Transactions v1 .
	Custom Segment	✓	You can use CRUD operations on custom segments. For details, see the help topic Using CRUD Operations on Custom Records, Custom Lists, and Custom Transactions v1 .

Record Category	Record Type	SOAP Web Services	REST Web Services
	Custom Transaction (instances)	✓	You can use CRUD operations on custom transactions. For details, see the help topic Using CRUD Operations on Custom Records, Custom Lists, and Custom Transactions v1 .
	Prompt	✓	✓
	Text Enhance Action	✓	✓
Entities	Competitor	—	✓
	Contact	✓	✓
	Customer	✓	✓
	Customer Status	✓	✓
	Employee	✓	✓
	Group	✓	—
	HCM Job	—	✓
	Job	✓	✓
	Lead	✓	—
	Other Name	—	✓
	Partner	✓	✓
	Project	✓	—
	Project Status	✓	✓
	Project Type	✓	✓
	Prospect	✓	—
	Vendor	✓	✓
	Win/Loss Reason	✓	—
File Cabinet	File	✓	—
	Folder	✓	—
Items	Assembly Item (including Lot Numbered and Serialized)	✓	✓
(Matrix items supported for applicable item types)	Description Item	✓	✓

Record Category	Record Type	SOAP Web Services	REST Web Services
	Discount Item	✓	✓
	Download Item	✓	✓
	Gift Certificate Item	✓	✓
	Inventory Item (including Lot Numbered and Serialized)	✓	✓
	Item Group	✓	✓
	Item Revision	✓	✓
	Kit Item	✓	✓
	Markup Item	✓	✓
	Non-inventory Item for Purchase	✓	✓
	Non-inventory Item for Resale	✓	✓
	Non-inventory Item for Sale	✓	✓
	Other Charge Item for Purchase	✓	✓
	Other Charge Item for Resale	✓	✓
	Other Charge Item for Sale	✓	✓
	Payment Item	✓	✓
	Service Item for Purchase	✓	✓
	Service Item for Resale	✓	✓
	Service Item for Sale	✓	✓
	Shipping Item	—	✓
	Subtotal Item	✓	✓
	Subscription Plan	—	✓
	Subscription Term	—	✓
Lists	Account	✓	✓
	Accounting Period	✓	✓
	Address	✓	In REST web services, you can interact with address data directly when you work with records
	Billing Account	✓	✓

Record Category	Record Type	SOAP Web Services	REST Web Services
	Billing Schedule	✓	✓
	Bin	✓	✓
	BOM	✓	✓
	BOM Revision	✓	✓
	Budget Category	✓	✓
	Class	✓	✓
	Consolidated Exchange Rate	✓	✓
	Currency	✓	✓
	Currency Rate	✓	✓
	Customer-Subsidiary Relationship	✓	✓
	Department	✓	✓
	Expense Category	✓	✓
	Fair Value Price	✓	✓
	Gift Certificate	✓	✓
	Inbound Shipment	✓	✓
	Inventory Detail	✓	—
	Inventory Number	✓	✓
	Location	✓	✓
	Manufacturing Cost Template	✓	✓
	Manufacturing Routing	✓	✓
	Merchandise Hierarchy Level	—	✓
	Merchandise Hierarchy Node	✓	✓
	Merchandise Hierarchy Version	—	✓
	Nexus	✓	✓
	Order Type	✓	—
	Payment Instruments:	✓	—
	■ General Token ■ Payment Card ■ Payment Card Token		

Record Category	Record Type	SOAP Web Services	REST Web Services
	Payroll Item	✓	✓
	Price Book	—	✓
	Price Plan	—	✓
	Revenue Recognition Schedule	✓	✓
	Revenue Recognition Template	✓	✓
	Sales Role	✓	✓
	Sales Tax Item	✓	✓
	Subsidiary	✓	✓
	Tax Group	✓	—
	Tax Type	✓	✓
	Unit of Measure	✓	✓
	Vendor-Subsidiary Relationship	✓	✓
Marketing	Campaign	✓	✓
	Campaign Response	✓	✓
	Coupon Code	✓	✓
	Email Template	—	✓
	Promotion	✓	✓
Other Lists	Contact Category	✓	✓
	Contact Role	✓	✓
	Cost Category	✓	✓
	Customer Category	✓	✓
	Customer Message	✓	✓
	Global Account Mapping	✓	✓
	Item Account Mapping	✓	✓
	Lead Source	✓	—
	Other Name Category	✓	✓

Record Category	Record Type	SOAP Web Services	REST Web Services
	Note Type	✓	—
	Partner Category	✓	—
	Payment Method	✓	—
	Price Level	✓	✓
	Pricing Group	✓	✓
	Tax Control Account	✓	—
	Term	✓	✓
	Vendor Category	✓	✓
	Win/Loss Reason	✓	—
Support	Issue	✓	✓
	Solution	✓	—
	Support Case	✓	✓
	Support Case Issue	✓	—
	Support Case Origin	✓	—
	Support Case Priority	✓	—
	Support Case Status	✓	—
	Support Case Type	✓	—
	Support Issue Type	✓	—
	Topic	✓	✓
Transactions	Advanced Intercompany Journal Entry	✓	✓
	Analytical Impact	—	✓
	Assembly Build	✓	✓
	Assembly Unbuild	✓	✓
	Bin Putaway Worksheet	✓	✓
	Bin Transfer	✓	✓
	Blanket Purchase Order	—	✓
	Budget	✓	—

Record Category	Record Type	SOAP Web Services	REST Web Services
	Cash Refund	✓	✓
	Cash Sale	✓	✓
	Change Order	—	✓
	Charge	✓	✓
	Check	✓	✓
	Credit Card Charge	—	✓
	Credit Card Refund	—	✓
	Credit Memo	✓	✓
	Customer Deposit	✓	✓
	Customer Payment	✓	✓
	Customer Refund	✓	✓
	Deposit	✓	✓
	Deposit Application	✓	✓
	Estimate	✓	✓
	Expense Report	✓	✓
	Fulfillment Request	—	✓
	Impact Subcategory	—	✓
	Intercompany Journal Entry	✓	✓
	Intercompany Transfer Order	✓	✓
	Inventory Adjustment	✓	✓
	Inventory Cost Revaluation	✓	✓
	Inventory Count	—	✓
	Inventory Transfer	✓	✓
	Invoice	✓	✓
	Item Demand Plan	✓	—
	Item Fulfillment	✓	✓
	Item Receipt	✓	✓
	Item Supply Plan	✓	—
	Journal Entry	✓	✓

Record Category	Record Type	SOAP Web Services	REST Web Services
	Landed Cost	✓	—
	Manufacturing Operation Task	✓	✓
	Opportunity	✓	✓
	Paycheck	✓	✓
	Paycheck Journal	✓	—
	Period End Journal	✓	✓
	Purchase Contract	—	✓
	Purchase Order	✓	✓
	Requisition	✓	✓
	Resource Group	✓	✓
	Return Authorization	✓	✓
	Sales Order	✓	✓
	Statistical Journal Entry	✓	✓
	Subscription	—	✓
	Subscription Line	—	✓
	Subscription Plan	—	✓
	Subscription Term	—	✓
	Time	✓	—
	Time Bill (Track Time)	✓	✓
	Transfer Order	✓	✓
	Usage	✓	✓
	Vendor Bill	✓	✓
	Vendor Credit	✓	✓
	Vendor Payment	✓	✓
	Vendor Prepayment	—	✓
	Vendor Prepayment Application	—	✓
	Vendor Return Authorization	✓	✓
	Weekly Timesheet	—	✓

Record Category	Record Type	SOAP Web Services	REST Web Services
	Work Order	✓	✓
	Work Order Close	✓	✓
	Work Order Completion	✓	✓
	Work Order Issue	✓	✓
Web Site	Commerce Category	—	✓
	Site Category	✓	—